DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA06 – DESIGN AND ANALYSIS OF ALGORITHMS

CO3 – CASE STUDY: DIVIDE-AND-CONQUER ENGINE OF A RETAIL DATA ANALYTICS PLATFORM

Student Details	Information
Student Name	Mohammed Affan M
Register Number	192572169
Assessment	CO3 – Case Study
Case Study	KartIQ Retail Data Analytics Platform
Mode	Individual

Aim: To redesign the four KartIQ modules using Merge Sort, Quick Sort, Binary Search and Strassen's matrix multiplication, with traces, recurrences, calculations and justified engineering decisions.

(a) Understand the Case

Module	Computational requirement	Key issues and consequence of a poor choice
1. Nightly order consolidation	Sort 10,000,000 orders by timestamp; equal timestamps must retain arrival order	Stability is essential. Unstable sorting can reorder same-second gateway orders and make au
2. Live seller leaderboard	Repeatedly sort 100,000 seller scores; only a few scores change between refreshes	Data is nearly sorted. First-element Quick Sort pivot can create 0 and n–1 partitions, causing
3. Order lookup	Find the first order with a requested timestamp in the sorted order list	Binary Search requires sorted data. Duplicates require a lower-bound/first-occurrence variant.
4. Recommendation scores	Multiply two 1,024 × 1,024 matrices.	Conventional multiplication needs n³ scalar multiplications. Strassen reduces recursive multipl

(b) Module 1 – Merge Sort for Nightly Order Consolidation

Divide, conquer and combine

Divide: Split the order list into two halves until each sublist contains one order. **Conquer:** Recursively sort each half by timestamp. **Combine:** Merge the two sorted halves by repeatedly selecting the order with the smaller timestamp. When timestamps are equal, select the item from the left half first. This preserves original arrival order and makes Merge Sort stable.

Sample input: (105,O1), (102,O2), (105,O3), (101,O4), (102,O5), (103,O6).

```
Initial:
[(105,O1), (102,O2), (105,O3), (101,O4), (102,O5), (103,O6)]

Divide:
Left  = [(105,O1), (102,O2), (105,O3)]
Right = [(101,O4), (102,O5), (103,O6)]

Sort left:
[(105,O1)] + [(102,O2), (105,O3)]
=> [(102,O2), (105,O1), (105,O3)]

Sort right:
[(101,O4)] + [(102,O5), (103,O6)]
=> [(101,O4), (102,O5), (103,O6)]

Final merge:
(101,O4), (102,O2), (102,O5), (103,O6),
(105,O1), (105,O3)
```

The equal timestamp orders remain in their original order: O2 precedes O5 at timestamp 102, and O1 precedes O3 at timestamp 105. This happens because the merge selects from the left run when keys are equal.

Recurrence and comparison estimate

For n orders, Merge Sort divides into two subproblems of size n/2 and performs Θ(n) work to merge:

```
T(n) = 2T(n/2) + cn,     T(1)=d
```

By the Master Theorem, a=2, b=2 and f(n)=Θ(n). Since n^(log■2)=n, this is Case 2: T(n)=Θ(n log■n). For n=10,000,000, log■n ≈ 23.2535. The rough n log■n scale is about 232.5 million; a standard worst-case comparison upper bound is n■log■n■–2^■log■n■+1 = 223,222,785 comparisons (ceil(log■n)=24). Actual comparisons depend on the order data and merge sizes.

Memory: Array-based Merge Sort normally requires Θ(n) auxiliary storage for merging, in addition to the input. If all orders do not fit in RAM, use **external Merge Sort**: read manageable chunks, sort each chunk in memory, write sorted runs to disk, then perform a multiway merge using buffers. This trades disk I/O for lower RAM use. The merge can preserve stability by carrying original arrival sequence numbers as a tie-breaker if runs are merged independently.

(c) Module 2 – Quick Sort for the Live Leaderboard

One partition trace using the current first-element pivot

Scores: [42, 17, 63, 8, 55, 29]. Choose the first element, 42, as pivot. Using a Hoare-style partition, move values smaller than 42 to the left and larger values to the right.

```
Start: [42, 17, 63, 8, 55, 29], pivot = 42
Scan from left: 63 is greater than pivot
Scan from right: 29 is smaller than pivot
Swap 63 and 29:
       [42, 17, 29, 8, 55, 63]
Continue scans; pointers cross.
Place pivot in its partition position:
       [8, 17, 29, 42, 55, 63]
Left partition:  [8, 17, 29]
Pivot:            42
Right partition: [55, 63]
```

Partition traces can vary slightly with the exact partition implementation. The important result is that the pivot separates smaller and larger scores.

Why the current pivot is slow; recurrence

With the first element as pivot, nearly sorted input often selects an extreme value. One partition then contains $n-1$ elements and the other contains zero. Repeating this creates deep recursion and quadratic comparisons, which explains the leaderboard freeze.

Best/balanced case: $T(n)=2T(n/2)+cn = \Theta(n \log n)$. **Worst case:** $T(n)=T(n-1)+cn = \Theta(n^2)$. The worst-case expansion is $cn+c(n-1)+\ldots+c = \Theta(n^2)$.

For 100,000 sellers already in ascending order, a first-element pivot can produce comparisons $99{,}999+99{,}998+\ldots+1 = n(n-1)/2 =$ **4,999,950,000**. This is an estimate for the standard extreme-pivot partition behaviour; exact counts depend on implementation.

Improved pivot strategy

Use median-of-three pivot selection (median of first, middle and last values) or a randomized pivot to reduce the chance of extreme partitions. For production robustness, use Introsort: begin with Quick Sort, monitor recursion depth, and switch to Heap Sort if depth exceeds a limit. Because the leaderboard is nearly sorted, also consider adaptive stable sorting such as Timsort, which can run in O(n) on already/nearly sorted data. The best choice depends on whether stability and update patterns matter. A balanced pivot alone improves expected behaviour but does not guarantee worst-case O(n log n); Introsort does.

(d) Module 3 – Binary Search for Order Lookup

Module 1 produces an array sorted by timestamp. This sorted order is essential: Binary Search relies on the fact that all smaller timestamps are on one side and all larger timestamps are on the other. If the list is not sorted, eliminating half the list is not logically valid.

Recurrence and maximum comparisons

Each comparison examines the middle element and discards approximately half the remaining elements:

```
T(n) = T(n/2) + 1,     T(1)=1
```

After k comparisons, the remaining search size is at most n/2^k. Therefore k is Θ(log■n). For n=10,000,000, log■n≈23.2535, so the maximum number of comparisons for ordinary binary search is ■log■n■+1 = **24**.

Return the first matching timestamp

Use a lower-bound search. When A[mid].timestamp is greater than or equal to the target, save the candidate if equal and continue searching left. This ensures the first occurrence is returned.

```
FIRST-ORDER(A, target):
    low = 0
    high = length(A) - 1
    answer = -1
    while low <= high:
        mid = (low + high) // 2
        if A[mid].timestamp >= target:
            if A[mid].timestamp == target:
                answer = mid
            high = mid - 1
        else:
            low = mid + 1
    return answer
```

The algorithm returns the first index with the timestamp, or –1 if no order has that timestamp. It takes O(log n) time and O(1) extra space. For 10,000,000 orders, the search needs at most about 24 timestamp comparisons.

(e) Module 4 – Strassen's Method for Recommendation Scores

For block matrices A=[[A11,A12],[A21,A22]] and B=[[B11,B12],[B21,B22]], conventional multiplication uses eight block multiplications. Strassen reduces these to seven.

Seven Strassen products and reconstruction formulas

```
M1 = (A11 + A22)(B11 + B22)
M2 = (A21 + A22)B11
M3 = A11(B12 - B22)
M4 = A22(B21 - B11)
M5 = (A11 + A12)B22
M6 = (A21 - A11)(B11 + B12)
```

```
M7 = (A12 - A22)(B21 + B22)

C11 = M1 + M4 - M5 + M7
C12 = M3 + M5
C21 = M2 + M4
C22 = M1 - M2 + M3 + M6
```

Verification using the supplied 2 × 2 matrices

A=[[1,7],[3,5]] and B=[[1,3],[7,5]]. Thus A11=1, A12=7, A21=3, A22=5; B11=1, B12=3, B21=7, B22=5.

Product	Calculation	Value
M1	(1+5)(1+5) = 6×6	36
M2	(3+5)×1 = 8×1	8
M3	1×(3–5) = 1×(–2)	–2
M4	5×(7–1) = 5×6	30
M5	(1+7)×5 = 8×5	40
M6	(3–1)(1+3) = 2×4	8
M7	(7–5)(7+5) = 2×12	24

Reconstruct:

```
C11 = 36 + 30 - 40 + 24 = 50
C12 = -2 + 40 = 38
C21 =  8 + 30          = 38
C22 = 36 - 8 - 2 + 8   = 34

C = [[50, 38],
     [38, 34]]
```

Conventional verification: C11=1×1+7×7=50; C12=1×3+7×5=38; C21=3×1+5×7=38; C22=3×3+5×5=34. Both methods produce the same result.

Recurrence and multiplication comparison for n=1,024

Strassen recurrence: $T(n)=7T(n/2)+\Theta(n^2)$. By the Master Theorem, T(n)=Θ(n^(log■7))≈Θ(n^2.807). Conventional multiplication takes $\Theta(n^3)$. Since 1,024=2^10, recursive Strassen down to 1×1 performs 7^10=282,475,249 scalar multiplications. Conventional multiplication performs $1{,}024^3$=1,073,741,824 scalar multiplications. Strassen therefore uses about 26.3% as many scalar multiplications, approximately 3.8 times fewer.

This count excludes additions/subtractions, data movement and recursion overhead. Strassen may be slower for small or medium matrices because it performs many matrix additions, allocates temporary blocks, uses extra memory and may have less favourable cache behaviour. Padding non-power-of-two dimensions can also waste work.

Practical hybrid: Recursively apply Strassen only while block size is above a measured threshold; below it, switch to a tuned conventional matrix multiplication routine. Use blocked/cache-aware conventional multiplication, avoid unnecessary temporary allocations, and pad only when the padding overhead is acceptable.

(f) Final Algorithmic Design and Justification

Module	Final choice	Time complexity	Extra memory	Trade-off and condition to change
1. Order consolidation	Stable external Merge Sort; sta[illegible]	O(n log n) comparisons; [illegible]	O(n) [illegible]	[illegible] extra RAM or disk I/Os but preserves equal-timestamp
2. Seller leaderboard	Adaptive stable sort (e.g. Timsort) [illegible]	Timsort O(n) on nearly sorted [illegible]	[illegible]	[illegible]
3. Order lookup	Lower-bound Binary Search o[illegible]	O(log n), at most about 24 comparisons for 10 mil[illegible]	O(1)	Requires sorted timestamps and returns first duplicate. If data is
4. Recommendation matrix	Hybrid Strassen + conventional (blocked) [illegible] threshold	O(n^2.807) multiplications [illegible]	Temporary matrix blocks, [illegible]	Fewer scalar multiplications but [illegible] additions and O(n²) worksp

Overall Conclusion

The KartIQ platform needs algorithms selected according to data characteristics, correctness requirements and production constraints. Stable Merge Sort preserves audit order; adaptive sorting or robust Quick Sort prevents leaderboard degradation; lower-bound Binary Search provides fast first-match lookup; and a hybrid Strassen method reduces multiplication work for sufficiently large dense matrices. No single algorithm is best in every situation: stability, memory, pivot quality, sorted-data assumptions, cache behaviour and practical overhead must all be considered.